

Sleep vs GPA in College: Predecir el Promedio Académico del Semestre

Emilio López
A01711977@tec.mx

ABSTRACT Este reporte muestra cómo construimos, a mano y con gradiente descendiente, un modelo de regresión lineal múltiple para predecir el GPA del semestre de estudiantes universitarios a partir de datos de sueño y variables demográficas, usando el dataset “*Sleep vs GPA in College*”. Durante la limpieza de datos encontramos y corregimos datos duplicados, una fuga de información y varias columnas redundantes, y comparamos dos formas de tratar los datos nulos en las variables de carga académica, uno fue imputarlos con la mediana y el otro fue simplemente quitar esas filas. El modelo final, con un split de 70% de training, 15% de validation y 15% de test, llegó a un R^2 de 0.458 en training, 0.445 en validación y 0.444 en test con la estrategia de imputación, superando en las tres particiones a la estrategia de eliminar las filas (R^2 de 0.430, 0.276 y 0.412). Como validación adicional, entrenamos un LinearRegression de scikit-learn sobre los mismos datos, que dio resultados casi idénticos a nuestra implementación a mano, y un RandomForestRegressor para explorar relaciones no lineales, que sobreajustó (R^2 de 0.916 en training contra 0.396 en test) sin superar al modelo lineal. Esto sugiere que el modelo capta una parte de la variabilidad del GPA, sobre todo gracias al GPA previo del estudiante, y que la relación en esencia es lineal. Finalmente, regularizamos el RandomForest limitando la profundidad de sus árboles (elegida por 5-fold cross-validation), lo que mejoró su R^2 de test de 0.396 a 0.411, aunque sin superar al modelo lineal.

1. Introducción

El sueño es uno de los factores que más se ha estudiado en relación con el rendimiento académico. Dormir poco o tener horarios de sueño muy irregulares se ha relacionado consistentemente con menos capacidad de concentración, memoria y desempeño académico en estudiantes universitarios. Pero el desempeño de un estudiante no depende solo de cómo duerme, cosas como su historial académico, cuántos créditos está cursando y su contexto demográfico también importan.

El objetivo de este trabajo es armar, desde cero y sin usar librerías de modelado como scikit-learn, un modelo de regresión lineal múltiple que prediga el GPA del semestre de un estudiante a partir de sus patrones de sueño, su historial académico y variables demográficas, y después usar scikit-learn para validar ese modelo y explorar si una relación no lineal captura más señal. La idea no es solo llegar a un modelo que funcione, sino mostrar todo el proceso de limpieza, transformación y modelado de datos, incluyendo decisiones importantes como detectar fugas de información y comparar distintas formas de tratar los datos nulos.

2. Descripción del Dataset

El dataset principal, *college_sleep_and_gpa.csv*, es de Kaggle [1] y tiene 634 filas, cada una es el registro de un estudiante en un semestre en específico. También hay un dataset de referencia, *study_cohort_reference.csv*, que describe las cohortes (*study*, *university*, *semester*,

cohort_code) y cuántos estudiantes tiene cada una.

2.1.- Features

El dataset tiene 22 columnas, entre las que destacan:

- **student_id**: identificador del estudiante (no único entre universidades).
- **cohort_code**: código de la cohorte (universidad + semestre + estudio).
- **gender, first_generation, underrepresented**: variables demográficas.
- **avg_sleep_hours, daytime_sleep_minutes, sleep_midpoint_minutes, bedtime_variability, nights_tracked_fraction**: variables de sueño.
- **prior_gpa**: GPA acumulado antes del semestre.
- **term_gpa**: GPA del semestre (**variable objetivo**).
- **gpa_change**: diferencia entre *term_gpa* y *prior_gpa*.
- **term_units, term_load_z**: carga académica del semestre (créditos y carga normalizada).

3. Extracción y Limpieza de Datos

3.1.- Duplicados

Al revisar duplicados por *student_id* encontramos varias coincidencias. Pero al investigar esas filas vimos que en realidad eran estudiantes de distintas universidades que por casualidad compartían el mismo número de id, no registros duplicados de verdad. Para confirmarlo armamos una llave compuesta, *student_uid* (*cohort_code* + *student_id*), y con esa sí dio cero duplicados, cada fila era efectivamente un estudiante distinto.

3.2.- Valores Nulos

Las columnas *gender*, *first_generation* y *underrepresented* tenían muy pocos nulos (menos del 1%), así que simplemente quitamos esas filas sin que afectara casi nada el tamaño del dataset.

term_units y *term_load_z*, en cambio, tenían bastantes más nulos. Como no estaba claro qué estrategia era mejor, decidimos hacer dos versiones del dataset y comparar cuál funcionaba mejor en el modelo final:

- **df_imputado** (626 filas): los nulos de *term_units*/*term_load_z* se rellenaron con la mediana de cada columna.
- **df_eliminado** (481 filas): se quitaron las filas que tenían nulos en esas columnas.

3.3.- Columnas Redundantes y Fuga de Información

Encontramos y quitamos 9 columnas por ser redundantes, estar derivadas de otras, o directamente filtrar información de la variable objetivo:

- **gpa_change**: es literalmente *term_gpa* - *prior_gpa*, o sea que es una fuga directa de la variable objetivo. Quitarla no era solo por cuestión de limpieza, era necesario para que el modelo no hiciera trampa.
- **study, university, semester**: con tablas de contingencia vimos que tienen un mapeo 1:1 con *cohort_code*, así que no aportaban nada nuevo.
- **avg_sleep_minutes, sleep_midpoint_clock, under_6h_sleep, sleep_bracket**: son la misma información en otro formato (cambio de unidad, de formato, o agrupada en rangos) de columnas que ya teníamos (*avg_sleep_hours* y *sleep_midpoint_minutes*).

- **student_id**: ya lo reemplazamos con *student_uid*, y como identificador puro no aporta nada al modelo.

4. Transformación de Datos

4.1.- Análisis de Correlación

Calculamos la matriz de correlación de Pearson sobre las variables numéricas de *df_imputado* (Figura 1). La variable más correlacionada con *term_gpa* fue, por mucho, *prior_gpa* ($r = 0.638$), seguida de *sleep_midpoint_minutes* ($r = -0.197$), *underrepresented* ($r = -0.173$) y *avg_sleep_hours* ($r = 0.206$). Esto deja claro que el historial académico previo es el mejor predictor lineal que tenemos, mientras que las variables de sueño, al contrario de lo que pensábamos, aportan algo pero mucho más débil.

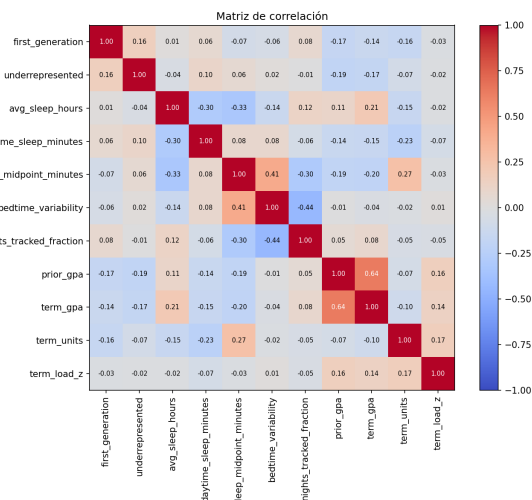


Figure 1: Matriz de correlación entre las variables numéricas del dataset.

La Figura 2 muestra cada predictor numérico contra *term_gpa* por separado, y confirma a simple vista que *prior_gpa* es la única variable con una tendencia lineal clara.

También comparamos *term_gpa* entre cohortes (Figura 3) y sí hay diferencias reales: las medias van de 3.28 a 3.66 según la universidad, así que vale la pena conservar *cohort_code* como predictor. En cambio, la diferencia de *term_gpa* por género fue casi nula (3.449 contra 3.452), así que esa variable probablemente no aporte mucho.

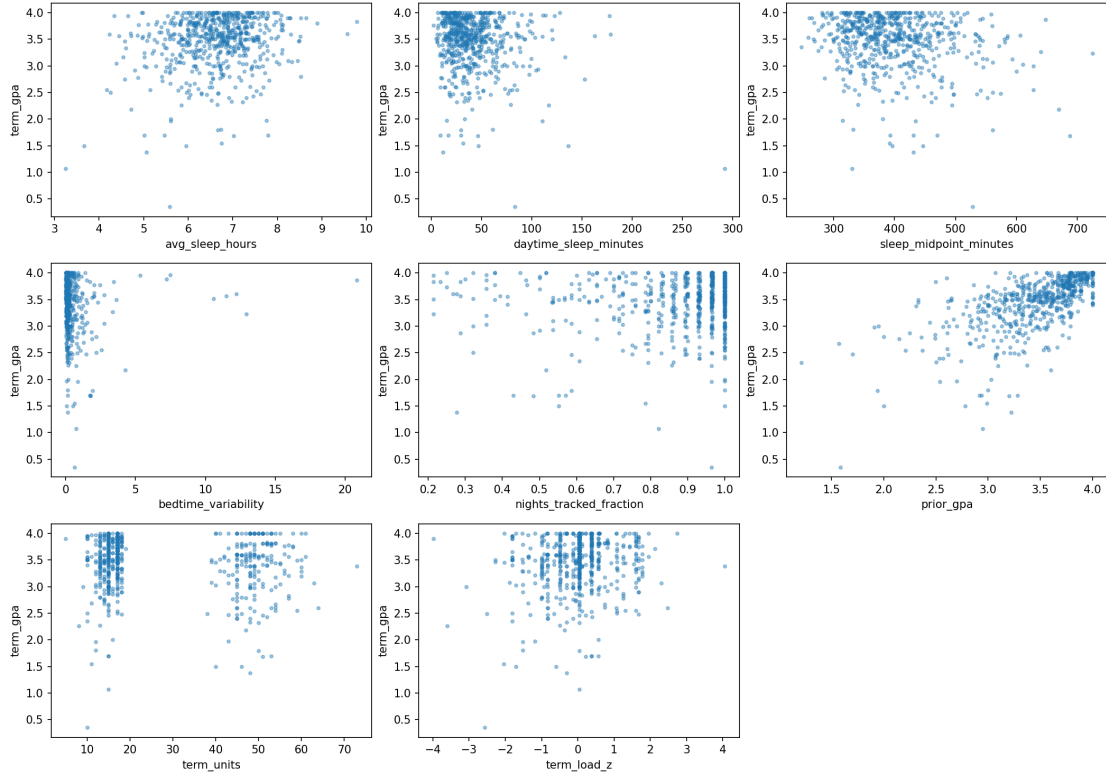


Figure 2: Dispersión de cada variable numérica contra *term_gpa*.

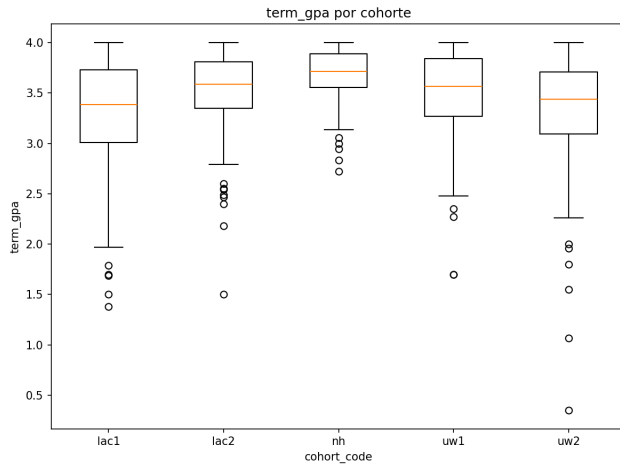


Figure 3: Distribución de *term_gpa* por cohorte.

4.2.- Codificación de Variables Categóricas

Las variables categóricas de texto (*gender* y *cohort_code*) se convirtieron a columnas binarias de 0 y 1 con *One-Hot Encoding*, usando *drop_first=True* para que las columnas dummy no queden perfectamente correlacionadas entre sí (si no, la regresión tendría columnas linealmente dependientes).

4.3.- Escalamiento

Antes de entrenar, estandarizamos todas las variables predictoras a media 0 y desviación 1. Importante: la media y desviación usadas para escalar el conjunto de prueba se calcularon solo con el conjunto de training, para no filtrar información del test hacia el entrenamiento.

4.4.- División del Dataset

Dividimos el dataset al azar en 70% para entrenamiento, 15% para validación y 15% para prueba, con una semilla fija para que sea reproducible. Elegimos darle la mayor parte a entrenamiento porque, después de la limpieza, el dataset ya es relativamente chico (626 filas en *df_imputado*, 481 en *df_eliminado*): con menos filas de train el modelo tendría menos información para ajustar sus parámetros, así que preferimos sacrificar algo de tamaño en validación y test a cambio de un conjunto de entrenamiento más grande. El set de validación nos sirve como una tercera métrica intermedia entre train y test, sobre todo para detectar si el modelo está sobreajustando antes de tocar el conjunto de prueba. Esto se hizo por separado tanto para *df_imputado* como para *df_eliminado*.

5. Construcción y Evaluación del Modelo

5.1.- Selección de Features y Variable Objetivo

Usamos todas las columnas que quedaron después de la limpieza (menos *term_gpa*) como predictoras, y *term_gpa* como variable objetivo. Entrenamos y evaluamos el modelo por separado en *df_imputado* y *df_eliminado*, para ver cómo afecta la estrategia de tratar nulos al resultado final.

5.2.- Hipótesis

La hipótesis del modelo es simplemente una combinación lineal de las predictoras más un término de sesgo (*bias*):

$$h(x) = x \cdot \theta + b \quad (1)$$

donde θ es el vector de coeficientes y b es el sesgo, que manejamos como un parámetro aparte en vez de agregar una columna de puros unos.

5.3.- Función de Costo

Usamos el Error Cuadrático Medio (MSE) como función de pérdida:

$$J(\theta, b) = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i)^2 \quad (2)$$

5.4.- Descenso de Gradiente

Los parámetros arrancan en cero y se van actualizando, iteración tras iteración, en dirección contraria al gradiente de la función de costo:

$$\theta := \theta - \alpha \cdot \frac{1}{m} X^T (h(X) - y) \quad (3)$$

$$b := b - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i) \quad (4)$$

con una tasa de aprendizaje $\alpha = 0.01$. El entrenamiento para cuando el R^2 de entrenamiento llega a 0.95 o cuando se cumplen 10,000 épocas (al momento de ejecutarlos, ambos modelos corrieron las 10,000 épocas completas sin poder llegar al umbral de 0.95). Estos tres valores (α , el umbral de R^2 y el límite de épocas) son los que se vieron en clase como referencia para este tipo de problema.

5.5.- Evaluación del Modelo

El desempeño se evaluó con R^2 y RMSE:

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}, \quad \text{RMSE} = \sqrt{\frac{\sum (y - \hat{y})^2}{m}} \quad (5)$$

6. Visualización y Resultados

La Tabla 1 nos muestra el desempeño final de ambas estrategias de tratamiento de datos nulos.

Dataset / Partición	R^2	RMSE
df_imputado – Train	0.458	0.368
df_imputado – Validate	0.445	0.354
df_imputado – Test	0.444	0.398
df_eliminado – Train	0.430	0.401
df_eliminado – Validate	0.276	0.403
df_eliminado – Test	0.412	0.467

Table 1: Comparación de desempeño entre las estrategias de imputación con la mediana y eliminación de las filas, en las tres particiones (train 70%, validate 15%, test 15%).

df_imputado le ganó a *df_eliminado* en las tres particiones. Lo más notable es que su desempeño se mantiene estable entre train, validate y test (R^2 de 0.458 a 0.444), mientras que *df_eliminado* tiene una caída fuerte justo en validación (R^2 de 0.430 a 0.276, para luego recuperarse a 0.412 en test), esto probablemente porque su conjunto de entrenamiento es más chico (336 filas contra 438) y el split de 15% para validación le deja muy pocas filas para que el resultado sea estable. Con esta evidencia, la imputación con la mediana es la estrategia recomendada por encima de eliminar las filas.

La Figura 4 muestra que los dos modelos convergen bastante rápido (antes de la época 500), y que la ventaja de *df_imputado* se mantiene estable durante todo el entrenamiento.

La Figura 5 muestra el comportamiento típico de un modelo con R^2 moderado: para GPAs bajos (1.0–2.5) el modelo sobreestima casi siempre, y para GPAs altos (cerca de 4.0) hay bastante dispersión. Esto tiene sentido con un modelo que capta la tendencia general pero le cuesta con los extremos, un indicio de underfitting moderado.

7. Validación con Framework

Como último paso, reimplementamos el problema con scikit-learn sobre *df_imputado* (la estrategia ganadora), usando exactamente los mismos splits de train/validate/test que ya se generaron para el modelo a mano, para que la comparación fuera limpia, si algún modelo de framework se ve mejor o peor, la diferencia viene del algoritmo y no de estar viendo datos distintos.

Elegimos dos modelos de librería con objetivos distintos. Primero, LinearRegression, que resuelve la misma regresión lineal, para validar que nuestra implementación

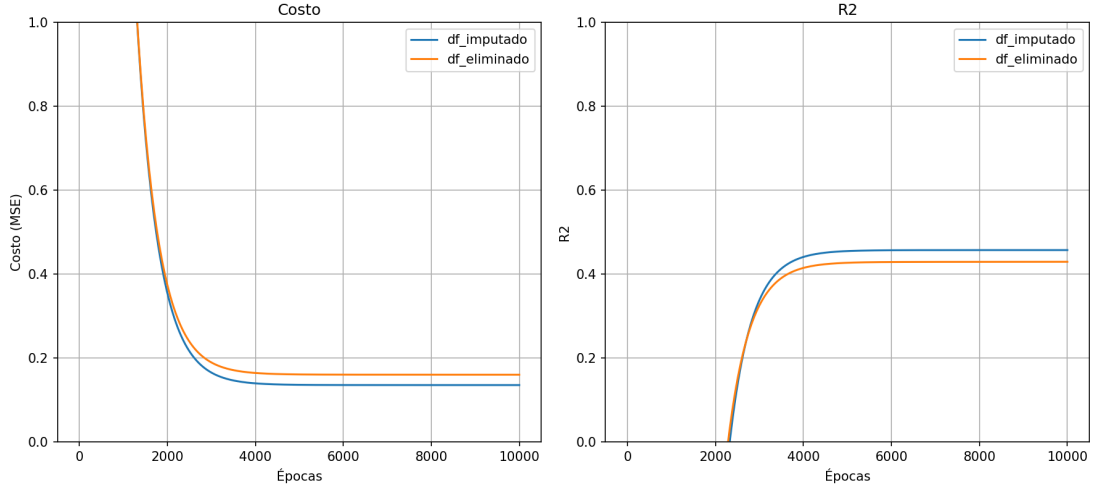


Figure 4: Curvas de costo y R^2 durante el entrenamiento, para ambas estrategias de imputación.

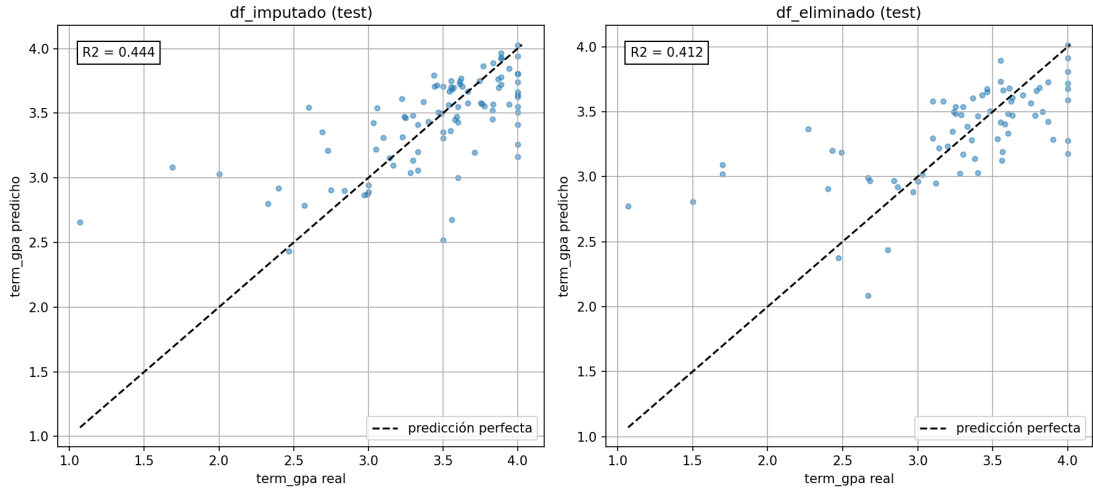


Figure 5: Predicción del modelo contra el valor real de *term_gpa* sobre el conjunto de prueba, para ambas estrategias.

manual efectivamente converge al mismo resultado que la solución cerrada. Segundo, RandomForestRegressor, un modelo no lineal basado en árboles, para explorar si hay relaciones entre las variables y *term_gpa* que un modelo lineal no puede capturar, utilizamos 300 árboles, que es un valor común que da una estimación estable sin un costo de cómputo alto para un dataset de este tamaño.

LinearRegression dio un R^2 de test de 0.444, prácticamente idéntico al 0.444 de nuestro modelo a mano, así que la implementación manual con descenso de gradiente sí está convergiendo al mismo resultado que la solución de scikit-learn.

RandomForestRegressor, en cambio, sobreajustó fuerte: R^2 de 0.916 en train contra apenas 0.396 en test, una brecha que indica que el modelo memorizó ruido del con-

Modelo / Partición	R^2	RMSE
LinearRegression – Train	0.460	0.367
LinearRegression – Validate	0.461	0.349
LinearRegression – Test	0.444	0.398
RandomForest – Train	0.916	0.145
RandomForest – Validate	0.487	0.341
RandomForest – Test	0.396	0.415

Table 2: Desempeño de LinearRegression y RandomForestRegressor sobre *df_imputado*

junto de entrenamiento en vez de aprender una relación que generalice. Ni en validación (0.487) ni en test (0.396) superó al modelo lineal.

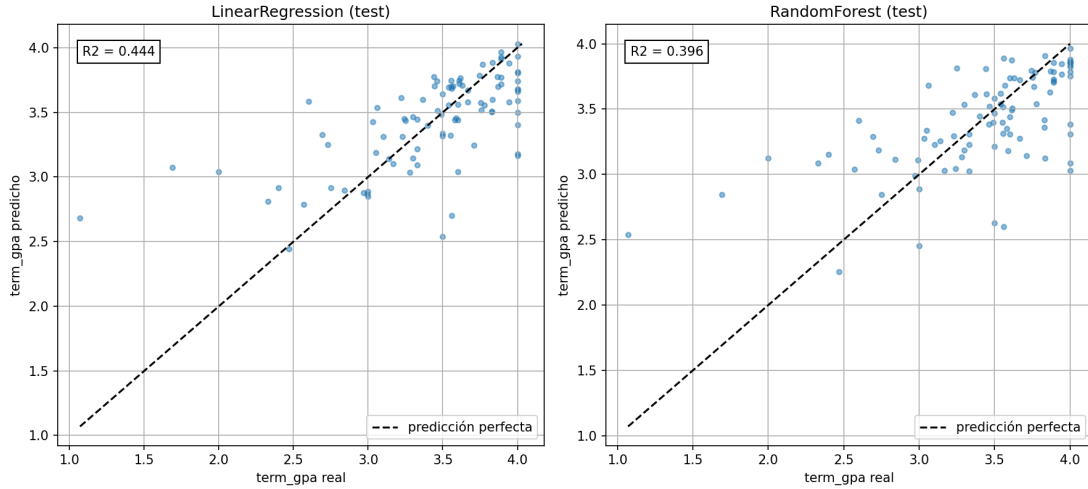


Figure 6: Predicción del modelo contra el valor real de *term_gpa* sobre el conjunto de prueba, para LinearRegression y RandomForest (*df_imputado*).

La Figura 6 muestra el mismo patrón que ya habíamos visto con el modelo a mano (Figura 5): dispersión considerable, sobre todo en los extremos, y ningún modelo se acerca mucho más a la diagonal que el otro. Con esta evidencia, concluimos que la relación entre estas variables y *term_gpa* es, en esencia, lineal, y que el techo de R^2 (0.44) probablemente viene de que los datos de sueño son ruidosos por naturaleza y no de que el modelo sea demasiado simple. Aumentar la complejidad del modelo, al menos con RandomForest, no ayuda y sí empeora la generalización.

7.1.- Regularización del RandomForest

En la sección anterior, el modelo lineal terminó superando al RandomForest tanto en validación como en test, y la sospecha principal es que el sobreajuste del RandomForest es lo que le impide generalizar tan bien como el modelo lineal. Antes de descartar al RandomForest, quisimos comprobar esa sospecha regularizándolo, para ver si, al reducir el sobreajuste, se acercaba más al desempeño del modelo lineal. Un RandomForest no es una regresión lineal con coeficientes que penalizar, es un ensamble de árboles de decisión, así que su equivalente conceptual a la regularización es limitar qué tan complejo puede ponerse cada árbol individual, por ejemplo con *max_depth*.

La primera decisión importante fue cómo elegir *max_depth*. Evaluar cada valor directamente contra el set de validación (94 filas) es poco confiable, ya que una muestra tan chica es ruidosa como criterio de selección. En su lugar, usamos 5-fold cross-validation sobre la unión de train y validation (532 filas), probando distintos valores de *max_depth* (dejando el resto de

los hiperparámetros del RandomForest original, como *n_estimators* = 300, sin tocar), promediando el R^2 de los 5 folds para cada valor, sin ver el conjunto de test en ningún momento de esta búsqueda.

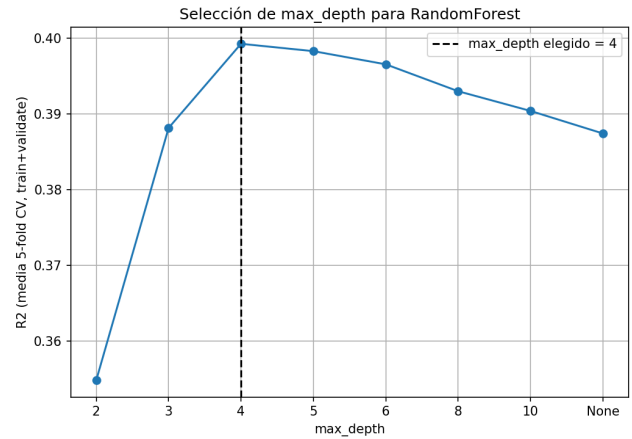


Figure 7: R^2 promedio de 5-fold cross-validation (train+validate) para cada *max_depth* probado en RandomForest.

La Figura 7 muestra un máximo claro en *max_depth*=4 (0.399 de R^2 promedio), muy por encima de dejar los árboles sin límite de profundidad (*max_depth*=None, 0.387). Con *max_depth*=4 elegido por Cross Validation, entrenamos sobre train y evaluamos en las tres particiones:

Limitar la profundidad de los árboles cierra buena parte de la brecha entre train y test (de 0.52 a 0.21), y el R^2 de test sube de 0.396 a 0.411, se sacrifica bastante ajuste sobre los datos ya vistos (train pasa de 0.916 a 0.616) a cambio que los árboles dejen de memorizar y

RF (max_depth=4) / Partición	R^2	RMSE
Train	0.616	0.310
Validate	0.456	0.351
Test	0.411	0.410

Table 3: Desempeño de RandomForest regularizado (max_depth=4) sobre df_imputado, elegido por 5-fold CV.

que generalizan mejor a datos nuevos. Aun así, sigue sin superar al modelo lineal (LinearRegression da 0.444 de test), lo que refuerza la conclusión de la sección anterior: la relación entre estas variables y *term_gpa* es en esencia lineal, y un modelo no lineal, aunque se regularice, no logra capturar información adicional real, solo deja de memorizar tanto ruido.

La Figura 8 muestra que el RandomForest regularizado predice de forma visiblemente más parecida al modelo lineal que el original, con menos puntos dispersos lejos de la diagonal.

Referencias

- [1] Feng, K. (n.d.). *Sleep vs GPA in College*. Kaggle. <https://www.kaggle.com/datasets/kylefengkfeng209/sleep-vs-gpa-in-college/data>

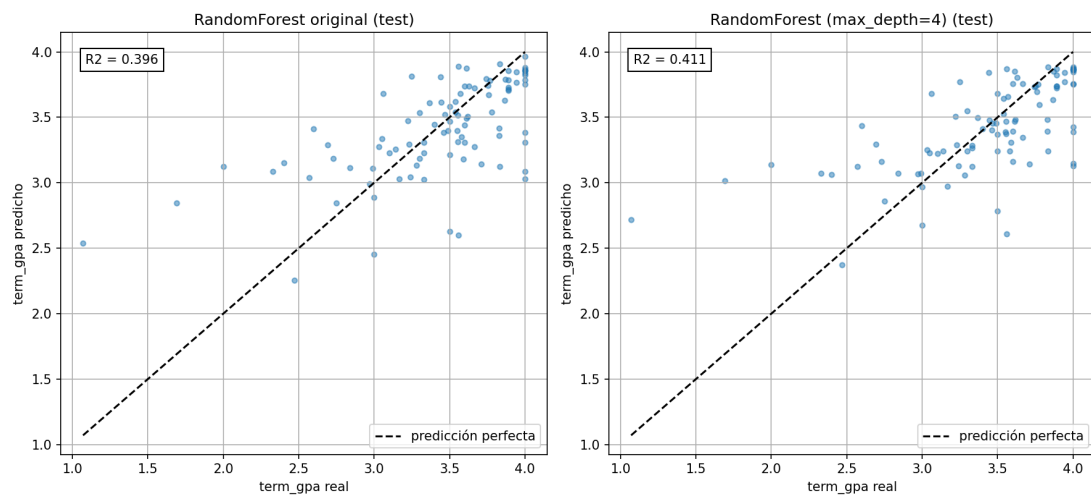


Figure 8: Predicción del modelo contra el valor real de *term_gpa* sobre el conjunto de prueba, para RandomForest original y regularizado ($\text{max_depth}=4$, df_imputado).